Student Performance Analytics System

Academic Record Management & Analytics Platform

UNSTRUCTURED DATABASE | ADAMAS UNIVERSITY

Team Members includes :

1. **Sk Asif Ahmed - UG/SOET/30/23/207**
2. **Amit Mishra - UG/02/BTCSE/2023/205**
3. **Prionti Majumdar - UG/02/BTCSE/2023/204**
4. **Tanmoy Banik - UG/SOET/30/23/206**

Made with GAMMA

Backend Overview

The backend is built on a robust, modern JavaScript stack designed for flexibility, performance, and ease of maintenance. It follows the MVC (Model-View-Controller) architectural pattern, ensuring a clear separation of concerns across all layers of the application.

Node.js

Runtime environment powering server-side logic with non-blocking I/O for high performance.

Express.js

Lightweight web framework for building RESTful APIs with clean routing and middleware support.

MongoDB

Document-oriented NoSQL database offering schema flexibility and horizontal scalability.

Mongoose

ODM library providing schema-based solutions to model application data with validation.

Core Features

- RESTful API endpoints for students, subjects, and marks
- MVC architecture for clean code organization
- Centralized error handling and input validation
- Modular route structure for easy extensibility
- Scalable design supporting future feature additions

Security & Data Integrity

- Input sanitization and schema-level validation via Mongoose
- Structured schema design enforcing data consistency
- Error middleware catching unhandled exceptions globally
- HTTP status codes returned accurately for all responses

The modular backend design allows individual components — such as authentication, analytics, or reporting — to be added or replaced without disrupting the rest of the system.

Made with GAMMA

Database Design & CRUD Operations

The system uses three primary MongoDB collections to organize academic data. Each collection is modeled with Mongoose schemas, enabling structured document storage while retaining the flexibility of a NoSQL database. All data operations follow standard CRUD patterns exposed through RESTful API endpoints.

students

Stores student profiles including roll number, name, semester, branch, and contact details. Each document is uniquely identified by a roll number or auto-generated `_id`.

subjects

Contains subject metadata such as subject code, name, credit hours, and associated semester. Linked to students via enrollment records.

marks

Records assessment scores for each student-subject pair. Includes internal marks, end-term marks, and total percentage for analytics computation.

CRUD Operations via RESTful Endpoints

1 Create

`POST /students` and `POST /marks` — Add new student records or insert mark entries. Input is validated against Mongoose schemas before insertion.

2 Read

`GET /students/:id` and `GET /marks/student/:id` — Fetch individual or aggregated student performance data. Supports query parameters for filtering by semester or branch.

3 Update

`PUT /marks/:id` — Modify existing mark records. Validation ensures updated values remain within acceptable ranges (e.g., 0–100).

4 Delete

`DELETE /students/:id` — Remove student records and associated marks. Cascading delete logic prevents orphaned documents in the `marks` collection.

Sample Query Structure: `db.marks.find({ studentId: "2021CSE001", semester: 3 }).sort({ total: -1 })` — Fetches all marks for a student in Semester 3, sorted by total score descending.

Made with GAMMA

Aggregation & Analytics

MongoDB's Aggregation Framework is the analytical engine of this system. It enables complex data transformations and computations directly within the database layer, reducing application-side processing and delivering fast, data-driven insights. The pipeline approach allows multiple stages to be chained together for powerful query composition.

Aggregation Operators Used

$match
Filters documents — e.g., students scoring above 75%.

$group
Groups documents by student or subject for aggregate calculations.

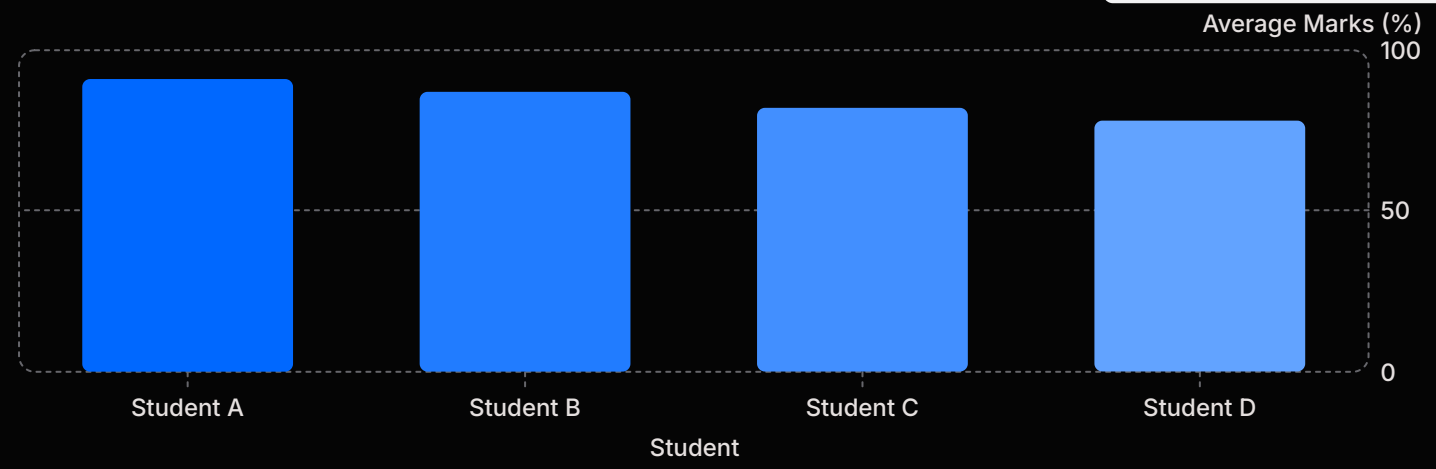
$avg
Computes average marks per student across all subjects.

$sort
Orders results — used to rank top performers.

Analytical Features Delivered

- **Average Marks Per Student** — Computes overall percentage across all enrolled subjects using `$group` and `$avg`.
- **Top 3 Performers** — Ranks students by total score and returns the top three using `$sort` and `$limit`.
- **High Achievers Filter** — Identifies students scoring above 75% using `$match` with a conditional expression.
- **Subject-wise Breakdown** — Aggregates scores grouped by subject to highlight class performance trends.

All aggregation pipelines execute on the database server, minimizing data transfer and maximizing query performance — a key advantage of MongoDB's native aggregation framework.



Sample visualization of student average marks — the aggregation pipeline computes these values server-side and returns them ready for frontend rendering.

Made with GAMMA

Frontend Overview

The frontend is built with React, providing a dynamic, component-driven user interface that communicates with the backend via RESTful API calls. The UI is designed for clarity and ease of use, ensuring that academic data is accessible and visually intuitive for both students and administrators.

Dashboard UI

A centralized home screen displaying key performance indicators, recent activity, and quick navigation to student records, subject lists, and mark sheets. Built with reusable React components for consistency.

Performance Visualization

Interactive charts and graphs render student performance data in real time. Bar charts, leaderboards, and percentage gauges provide at-a-glance insights into academic trends.

Responsive Design

The interface adapts seamlessly across desktop, tablet, and mobile viewports. CSS Grid and Flexbox ensure layout integrity regardless of screen size, supporting accessibility standards.

Intuitive Navigation

A minimal navigation structure with clear routes — Students, Subjects, Marks, Analytics — reduces cognitive load. Breadcrumbs and active-state indicators help users orient themselves within the app.

React

Component-based UI library enabling modular, reusable interface elements.

Axios / Fetch API

HTTP client for seamless communication with backend RESTful endpoints.

Chart Library

Lightweight visualization library (e.g., Recharts or Chart.js) for rendering analytics.

CSS / Styled Components

Modern styling approach ensuring consistent, maintainable UI design.

Made with GAMMA

Conclusion & Key Outcomes

The **Student Performance Analytics System** successfully demonstrates a full-stack approach to academic data management — from structured NoSQL data modeling to real-time analytical insights and a polished user interface. The project meets all core requirements of the Unstructured Database course while delivering a production-ready architecture.

Efficient Data Management

MongoDB collections and Mongoose schemas provide a clean, validated structure for storing and retrieving student, subject, and mark records with full CRUD support.

Real-Time Analytics

MongoDB's Aggregation Framework delivers instant insights — average scores, top performers, and high-achiever filters — computed server-side for optimal performance.

Scalable System Design

The modular MVC backend and component-based React frontend are designed to scale. New features — authentication, reporting, notifications — can be integrated without restructuring existing code.

> **Thank You** — We appreciate your time and attention. This project reflects our team's understanding of unstructured database systems, RESTful API design, and full-stack development principles. We welcome any questions or feedback.

ADAMAS UNIVERSITY | UNSTRUCTURED DATABASE | 2025–26

Made with GAMMA